

Practical Work and Record: Computational Analysis and Figure-Generation Protocol

MSc Artificial intelligence for drug discovery QMUL

JungHyun(Andy) Bang

Before You Begin

This protocol is intended for students and researchers who wish to reuse this pipeline in future studies.

If some parts are not explained in detail, please refer to the individual md files included in the subdirectories which will contain the indepth explanation for each of the code blocks or modules.

The pipeline source code is version-controlled and publicly available on GitHub:

```
https://github.com/JHBH9e8at/DProject_Workspace
```

The production deployment used to generate the reported results resides on the department server, under the canonical layout already introduced in Section 1:

```
/home/andy/proj/  
├─ DProject_Workspace/  # cloned copy of the GitHub repository above  
└─ work_results/       # inputs and generated results
```

Server access

The server (**hal**) is not directly reachable from the open internet and is accessed through an SSH jump host (**qmul**). The relevant `~/.ssh/config` entries are:

```
Host qmul  
  HostName students3.ph.qmul.ac.uk  
  User ec25003  
  
Host hal  
  HostName 138.37.52.153  
  User andy  
  ProxyJump qmul  
  IdentityFile ~/.ssh/id_ed25519_qmul_jhbk  
  IdentitiesOnly yes
```

With this configuration in place, the server is reached with a single command:

```
ssh hal
```

`ssh` transparently establishes the connection through `qmul` and lands directly in the production account on `ha1`. Access requires a QMUL account authorised to reach `students3.ph.qmul.ac.uk` and the corresponding SSH key pair; neither the key nor QMUL credentials are included in this repository or record.

For students and future users

Long story short, the pipeline is designed to run with minimal manual intervention once the required data have been placed in the correct locations. Most intermediate analysis tables, metadata files, manifests, and figures are generated automatically by the pipeline.

Do not rely only on the final figures when interpreting the results. The corresponding manifests, metadata files, and analysis tables contain the exact sample counts, calculation settings, input paths, and numerical results used to generate each figure.

Please read the configuration and troubleshooting sections carefully before running the pipeline. Most path, run-ID, cache, and missing-input problems can be avoided by checking the `.in` files before execution.

For markers and reviewers

The principal numerical results, metadata, manifests, and figures are stored under:

```
/home/andy/proj/work_results
```

The following table provides a quick index to the principal inputs and outputs used by the current workflow.

Result category	Location relative to <code>work_results/</code>	Contents
Completed AHC inputs	<code>AHC_Resultsbin/SMILES-RNN_PPSrun/</code> and <code>AHC_Resultsbin/SMILES-RNN_PRun/</code>	Original PPS and PR AHC results, including <code>scores.csv</code>
AHC monitoring	<code>AHC_monitoring/PPS/</code> and <code>AHC_monitoring/PR/</code>	Merged completed iterations, cleaned populations, physicochemical plots, and docking-score trajectories
Cross-docking calculation	<code>crossdocking/</code>	Filtered populations, job-level docking results, double-arm results, and calculation manifests
Cross-docking completion record	<code>analysis/completed_crossdocking/CDres_LP_ON/</code>	Completion metadata and <code>crossdocking_analysis.json</code>
Cross-docked interactions	<code>analysis/crossdocking_interactions/CDres_LP_ON/</code>	PoseCheck and ProLIF results for the cross-docked populations
Reference interactions	<code>analysis/reference_interactions/CDres_LP_ON/</code>	ProLIF interactions calculated for the PPS and PR reference ligands

Result category	Location relative to <code>work_results/</code>	Contents
Chemical-space analysis	<code>analysis/chemical_space/PR_PPS_completed_chemical_space/</code>	UMAP/t-SNE coordinates, cached embeddings, membership information, and metadata
Scaffold analysis	<code>analysis/scaffold_diversity/</code>	Scaffold-frequency and fingerprint-diversity results
Residue-hotspot analysis	<code>analysis/residue_hotspot/PR_PPS_analysis_residue_hotspot/</code>	Residue and interaction-type prevalence tables
Pocket-volume analysis	<code>analysis/pocket_volume/</code>	Pocket-volume estimates and calculation metadata
Redocking validation	<code>analysis/redocking/</code>	PPS/PR docking-score and RMSD summaries
Final figures	<code>Figs/</code>	Figures generated by all enabled plotting blocks

The most important files for reviewing the calculations are described below.

Cross-docking calculation summary

```

/home/andy/proj/work_results/crossdocking/
├── pipeline_run_summary.json
├── crossdocking_calculation_manifest.json
├── filtering/
│   ├── PPS_filtered.csv
│   ├── PPS_filtered_filterlog.txt
│   ├── PR_filtered.csv
│   └── PR_filtered_filterlog.txt
└── crossdocking/
    ├── double_arm_manifest.csv
    ├── double_arm_summary.csv
    ├── double_arm_docking_scores.csv
    └── double_arm_best_docking_scores.csv

```

`pipeline_run_summary.json` and `crossdocking_calculation_manifest.json` record the execution status and calculation context. The filtering logs report the numbers of molecules retained and removed at each preprocessing step.

Cross-docking interaction results

```

/home/andy/proj/work_results/analysis/
├── completed_crossdocking/CDres_LP_ON/
│   └── crossdocking_analysis.json
├── crossdocking_interactions/CDres_LP_ON/
│   ├── populations/
│   └── aggregate/

```

```
├── interactions_long.csv
├── pose_metadata.csv
└── reference_interactions/CDres_LP_ON/
    └── reference_interactions.csv
```

interactions_long.csv contains the individual protein-ligand interaction records used for population-level analysis. **pose_metadata.csv** identifies the analysed molecules and poses. **reference_interactions.csv** contains the interactions calculated for the PPS and PR reference ligands.

Residue-hotspot results

```
/home/andy/proj/work_results/analysis/residue_hotspot/
├── PR_PPS_analysis_residue_hotspot/
│   └── tables/
│       ├── residue_prevalence.csv
│       └── interaction_type_prevalence.csv
```

These tables contain the numerical values underlying the interaction-hotspot figures. **residue_prevalence.csv** reports residue-level prevalence, whereas **interaction_type_prevalence.csv** separates the results by interaction type.

Chemical-space results

```
/home/andy/proj/work_results/analysis/chemical_space/
├── PR_PPS_completed_chemical_space/
│   └── tables/
│       └── joint_umap_fp.csv.gz
```

The compressed CSV contains the calculated joint chemical-space coordinates and population membership used to generate the corresponding plots.

Final figures

```
/home/andy/proj/work_results/Figs/
```

Figures are grouped by analysis function and run ID. Each figure should be interpreted together with its corresponding numerical table and manifest. Where available, the manifest records the source files, settings, run ID, and output paths used for figure generation.

The principal colour convention used throughout the comparative figures is:

```
PPS: red   (#D9534F)
PR:  blue  (#2878B5)
```

The output directory may contain results from multiple executions. Reviewers should use the run ID, manifest, and file modification time to identify the outputs belonging to the reported production run.

1. Purpose and scope

This protocol describes the computational workflow used to analyse completed Augmented Hill-Climb (AHC) molecular-generation results, perform state-specific cross-docking, evaluate protein-ligand interactions and structural properties, and generate the final analysis figures.

The workflow compares molecules generated against two conformational states:

- **PPS:** post-powerstroke state
- **PR:** pre-recovery state

The repository contains the monitoring, analysis, cross-docking, and figure-generation code. The external AHC molecular-generation engine itself is not included.

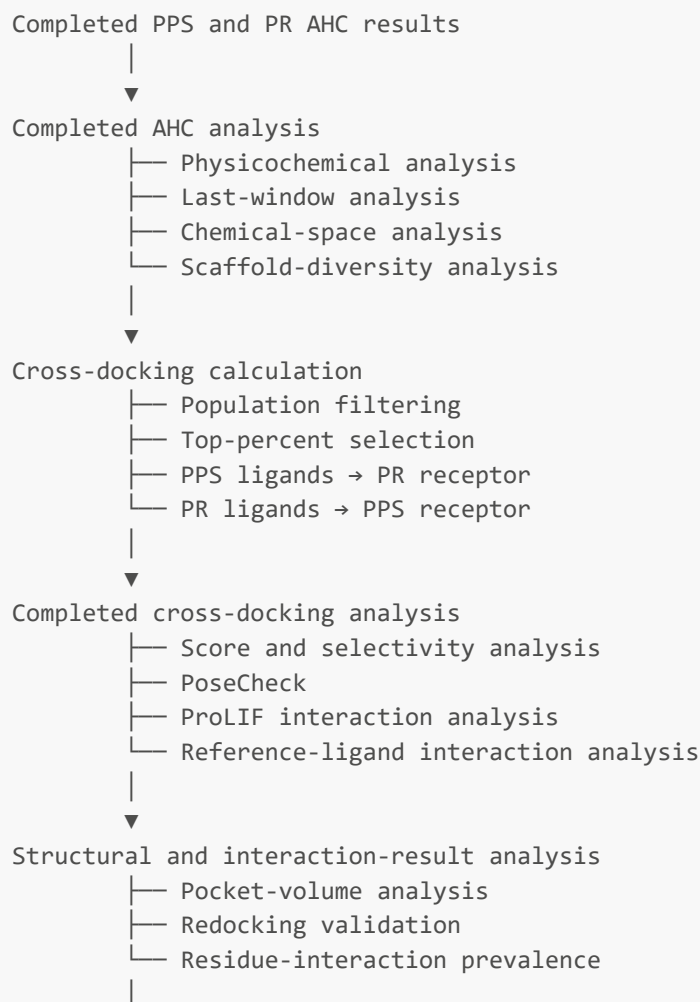
The canonical server layout assumed in this protocol is:

```
/home/andy/proj/  
├─ DProject_Workspace/      # Git-controlled source code  
└─ work_results/           # Input data and generated results
```

Scripts inside **blocks/** directories are internal implementation units. Users should normally interact only with the top-level runners and their corresponding **.in** configuration files. Files under **bins/** are legacy or archived material and are not part of the production workflow.

2. Workflow overview

The completed-data workflow is executed in the following order:



Final figure generation

AHC monitoring is an independent optional workflow:

```
Active AHC run
├─ Iteration CSV monitoring
│   ├── Merge completed iterations
│   ├── Clean the population
│   ├── Generate physicochemical figures
│   └─ Generate a docking-score trajectory
```

Monitoring is not a prerequisite for completed AHC analysis but since the AHC took around 16~18 days, it was developed as a tool that can conduct a mid point or a routine check to identify how the model is running and to make decisions regarding when to stop the model. - Specifically, the Generate a docking-score trajectory figure were visualised to check the plateauing of the docking scores to make such decisions. The examples of past mid point checking can be found in the subdirectories contained in `/home/andy/Bin_backup/701/code/developing_dir/tempdevtest` where the figures are in the subdirectory named figure

ie)
`/home/andy/Bin_backup/701/code/developing_dir/tempdevtest/mp20260720/figures/Docking_Score_Trajectory_PR_PR_241_20260720_115055PR_cleaned_20260720_115125.png`

I believe this will be a very useful tool for the person continuing this project.

Part I. Requirements

3. Software requirements

3.1 Operating environment

The production workflow is intended for a Linux server. The currently tested environment includes Python 3.10 and Schrödinger 2024-2.

```
Repository: /home/andy/proj/DProject_Workspace
Results root: /home/andy/proj/work_results
```

3.2 Python dependencies

Package	Principal use
pandas	CSV ingestion, filtering, aggregation, and output
NumPy	Numerical operations
RDKit	SMILES parsing, canonicalisation, descriptors, fingerprints, scaffolds, and structural alerts
matplotlib	Figure generation
Pillow	Image composition and font handling

Package	Principal use
scikit-learn	Scaling and t-SNE
umap-learn	UMAP chemical-space embedding
MDAnalysis	Protein and ligand structure handling
ProLIF	Protein-ligand interaction fingerprints
PoseCheck	Docked-pose quality assessment
SciPy	Numerical and statistical support

Some legacy files may import additional packages such as Dask or seaborn. Files under `bins/` should not be used to determine the minimum production environment.

The environment that the pipeline was tested and run can be found in

```
/home/andy/proj/envs
```

where Two analysis environments are stroed.

```
analysisenv # General AHC, structural, and figure calculations
interanaly  # MDAnalysis, ProLIF, and PoseCheck calculations
```

3.3 Schrödinger requirements

Cross-docking requires valid Schrödinger and Glide installations and an available licence. The `SCHRODINGER` environment variable must point to the installation directory. - this should be provided by the department.

4. Required user-provided data

4.1 Completed AHC score tables

```
/home/andy/proj/work_results/AHC_Resultsbin/
├── SMILES-RNN_PPSrun/
│   └── scores.csv
└── SMILES-RNN_PRRun/
    └── scores.csv
```

The tables must contain the molecular identity, SMILES, validity, uniqueness, iteration, docking-score, and best-variant information expected by the AHC analysis code. Zero docking-score rows are treated as placeholders or unusable docking results by the relevant analysis blocks.

4.2 AHC docking pose files

Cross-docking with `ligand_prep_mode=off` requires the original AHC pose hierarchy:

```
SMILES-RNN_PPSrun/PPS_GlideDock/<iteration>/<variant>_lib.sdfgz
SMILES-RNN_PRRun/PR_GlideDock/<iteration>/<variant>_lib.sdfgz
```

With `ligand_prep_mode=on`, ligand structures are prepared from filtered SMILES using LigPrep. The AHC score tables remain required.

4.3 Monitoring iteration files

Monitoring an active run requires at least two iteration CSVs:

```
SMILES-RNN_PRRun/iterations/
├── 000001_scores.csv
├── 000002_scores.csv
└── ...
```

The newest file is deliberately excluded because the AHC process may still be writing it. Fixed-width, zero-padded names preserve chronological sorting.

4.4 Receptors, reference ligands, and Glide grids

```
AHC_Related/ref_structures/
├── Structures/
│   ├── PPS_Protein_no_conect.pdb
│   ├── PR_Protein_no_conect.pdb
│   ├── PPS_ligand.sdfgz
│   ├── PR_ligand.sdfgz
│   └── ligand_20W.pdb
├── grids/
│   ├── PPS/PPSgrid.zip
│   └── PR/PR_Auto_Grid.zip
└── ref_desc.csv
```

The grid files must be prepared from the intended receptor structures and docking-site definitions.

4.5 Fpocket results

```
/home/andy/proj/work_results/fpocket_res/
├── PPS_Protein_out/
├── PPS_reori_out/
└── PR_potein_out/
```

The current hotspot configuration refers to:

```
PPS_Protein_out/pockets/pocket2_atm.pdb
PPS_Protein_out/pockets/pocket38_atm.pdb
PPS_Protein_out/pockets/pocket51_atm.pdb
PR_potein_out/pockets/pocket11_atm.pdb
```

```
PR_potein_out/pockets/pocket15_atm.pdb
PR_potein_out/pockets/pocket60_atm.pdb
```

Pocket numbers are specific to one Fpocket run and are not portable biological identifiers. If Fpocket is rerun or a receptor structure changes, the user must inspect the new pockets and update all affected `.in` files.

4.6 Redocking results

```
/home/andy/proj/work_results/Redocking/
├── PPS_docking.csv
├── PPS_RMSD.csv
├── PR_docking.csv
└── PR_RMSD.csv
```

These are user-provided upstream results. The workflow analyses and visualises them but does not perform the redocking calculation itself.

5. Pipeline-generated inputs

The following files are generated by the pipeline and should not be manually prepared for a complete run:

```
reference_interactions.csv
interactions_long.csv
pose_metadata.csv
residue_prevalence.csv
interaction_type_prevalence.csv
joint_umap_fp.csv.gz
crossdocking_analysis.json
```

For example:

```
Reference receptor + reference ligand
  → ProLIF analysis
  → reference_interactions.csv
```

Part II. Directory and configuration rules

6. Expected directory layout

```
/home/andy/proj/
├── DProject_Workspace/
│   ├── AHC_Related/
│   └── Modules/
│       ├── ACHresultspackage/
│       ├── Crossdocking/
│       └── Analysis_Figen_block/
└── work_results/
```

```
|— AHC_Resultsbin/  
|— AHC_monitoring/  
|— crossdocking/  
|— fpocket_res/  
|— Redocking/  
|— analysis/  
|— Figs/
```

The repository and calculation outputs are intentionally separated:

- `DProject_Workspace` is Git-controlled.
- `work_results` is not Git-controlled.
- Server-specific paths are configured in `.in` files.
- `blocks/` contains internal implementation code.
- `bins/` is not part of production execution.

Run commands from the repository root:

```
cd /home/andy/proj/DProject_Workspace
```

7. General `.in` configuration rules

Configuration files use one `key=value` assignment per line. Blank lines and lines beginning with `#` are ignored.

```
results_root=/home/andy/proj/work_results  
run_id=PR_PPS_analysis  
resume=OFF
```

Important rules:

1. `results_root` must identify the intended `work_results` directory.
2. `run_id` links upstream and downstream results.
3. Run IDs and paths are case-sensitive on Linux.
4. A downstream configuration must use the exact upstream run ID.
5. Enabled options may introduce additional required inputs.
6. Disabled blocks ignore their block-specific settings.
7. Relative paths are resolved according to the runner, normally from the configuration directory.
8. Server-specific absolute paths must be reviewed after moving the repository.

For example, these identifiers are different on Linux:

```
CDres_LP_ON  
CDres_LP_On
```

Part III. Running the pipeline

8. Preflight checks

```
cd /home/andy/proj/DProject_Workspace

test -f /home/andy/proj/work_results/AHC_Resultsbin/SMILES-RNN_PPScrun/scores.csv
test -f /home/andy/proj/work_results/AHC_Resultsbin/SMILES-RNN_PRRun/scores.csv
test -f AHC_Related/ref_structures/grids/PPS/PPSgrid.zip
test -f AHC_Related/ref_structures/grids/PR/PR_Auto_Grid.zip
test -f AHC_Related/ref_structures/Structures/PPS_Protein_no_conect.pdb
test -f AHC_Related/ref_structures/Structures/PR_Protein_no_conect.pdb
```

Use `dry_run=ON` in `Modules/Analysis_Figen_block/analysis.in` to validate and print the top-level analysis plan without starting calculations.

9. Step 0: running the upstream AHC generation

9.1 Scope and responsibility

The AHC generation engine is an external dependency and is not implemented in this repository. This repository provides the MolScore JSON files and Glide templates used to configure the PR and PPS runs:

```
AHC_Related/Runinputs/
├── PR/
│   ├── PR_F.json
│   └── PRGlide.in
└── PPS/
    ├── PPS_base.json
    └── PPSGlide.in
```

The current launch command expects an external SMILES-RNN installation at:

```
/home/andy/Project/MolScore_related/SMILES-RNN-main
```

Before attempting an AHC run, obtain the correct SMILES-RNN/MolScore installation, trained prior, environment, and local execution guidance from the researcher or supervisor responsible for maintaining the AHC environment.

9.2 Files and values to review before execution

The following values are specific to the current server and must be checked:

Setting	Current value	Required action
SMILES-RNN program	<code>/home/andy/Project/MolScore_related/SMILES-RNN-main/reinforcement_learning.py</code>	Confirm that the external program exists
Prior model	<code>/home/andy/Project/MolScore_related/SMILES-RNN-main/priors/ChEMBL28pur.ckpt</code>	Confirm that the intended prior checkpoint exists
Conda environment	<code>molscore</code>	Confirm that SMILES-RNN, MolScore, Dask, and docking dependencies are installed

Setting	Current value	Required action
Dask scheduler	<code>tcp://138.37.52.153:8786</code>	Confirm that the host address and port are correct for the server
Dask dashboard	<code>:8787</code>	Change if this port is already occupied
Dask local directory	<code>/home/andy/proj/701/output/dask_cache</code> in the historical command	Replace with an existing writable scratch/cache directory for the current server
AHC output root	<code>/home/andy/proj/work_results/AHC_Resultsbin</code>	Confirm that sufficient disk space is available
PR Glide grid	<code>AHC_Related/ref_structures/grids/PR/PR_Auto_Grid.zip</code>	Confirm that this is the intended PR grid
PPS Glide grid	<code>AHC_Related/ref_structures/grids/PPS/PPSgrid.zip</code>	Confirm that this is the intended PPS grid

The Dask address is stored in both AHC JSON files:

```
"cluster": "tcp://138.37.52.153:8786"
```

The output root and task name determine the final run directories. For example, the PR configuration contains:

```
{
  "task": "SMILES-RNN_PRun",
  "output_dir": "/home/andy/proj/work_results/AHC_Resultsbin"
}
```

This is expected to produce:

```
/home/andy/proj/work_results/AHC_Resultsbin/SMILES-RNN_PRun/
```

The PPS configuration uses `SMILES-RNN_PPSrun` in the same output root.

The Glide template path in each JSON file must point to its matching template:

```
// PR_F.json
"glide_template": "/home/andy/proj/DProject_Workspace/AHC_Related/Runinputs/PR/PRGlide.in"

// PPS_base.json
"glide_template":
"/home/andy/proj/DProject_Workspace/AHC_Related/Runinputs/PPS/PPSGlide.in"
```

Note: the snippets above are shown separately for explanation. JSON does not permit `//` comments; do not add those comments to the actual JSON files.

Each Glide template must also reference the correct receptor grid. For example:

```
# PRGlide.in
FORCEFIELD  OPLS4
GRIDFILE
/home/andy/proj/DProject_Workspace/AHC_Related/ref_structures/grids/PR/PR_Auto_Grid.zip
LIGANDFILE  ligand_file.sdf
PRECISION   SP
```

`ligand_file.sdf` is a placeholder substituted by the scoring workflow and should not be replaced with one fixed ligand file.

9.3 Start the Dask scheduler and workers

AHC runs are long-running jobs. A terminal multiplexer such as `screen` should be used so that the process remains active after disconnecting from the server.

```
screen -S ahc_run
conda activate molscore
```

Create a writable Dask cache directory. The following is an example; select a location appropriate for the server:

```
mkdir -p /home/andy/proj/temp/dask_cache
```

Start the scheduler:

```
dask-scheduler --port 8786 --dashboard-address :8787 &
```

Start the workers using the same address configured in `PR_F.json` and `PPS_base.json`:

```
dask-worker tcp://138.37.52.153:8786 \
--workers 8 \
--nthreads 1 \
--local-directory /home/andy/proj/temp/dask_cache &
```

The worker count must be selected according to the available CPU, memory, Schrödinger licence capacity, and site policy. Eight workers reproduce the historical command but should not be assumed to be universally appropriate.

Confirm that the processes are running:

```
ps -ef | grep -E "dask-(scheduler|worker)|dask scheduler|dask worker" | grep -v grep
ss -ltnp | grep -E ' :8786|:8787'
```

9.4 Run the PR AHC job

```
python /home/andy/Project/MolScore_related/SMILES-RNN-main/reinforcement_learning.py \  
-p /home/andy/Project/MolScore_related/SMILES-RNN-main/priors/ChEMBL28pur.ckpt \  
-m /home/andy/proj/DProject_Workspace/AHC_Related/Runinputs/PR/PR_F.json \  
--model RNN \  
AHC
```

The PR JSON uses the score metric:

```
PR_r_i_docking_score
```

with a minimisation objective.

9.5 Run the PPS AHC job

Run PPS in a separate **screen** session and, if required by the local execution policy, with its own scheduler, worker set, ports, and cache directory.

```
python /home/andy/Project/MolScore_related/SMILES-RNN-main/reinforcement_learning.py \  
-p /home/andy/Project/MolScore_related/SMILES-RNN-main/priors/ChEMBL28pur.ckpt \  
-m /home/andy/proj/DProject_Workspace/AHC_Related/Runinputs/PPS/PPS_base.json \  
--model RNN \  
AHC
```

The PPS JSON uses:

```
PPS_r_i_docking_score
```

with a minimisation objective.

Do not launch PR and PPS simultaneously against the same Dask scheduler or ports unless that arrangement has been deliberately configured and sufficient Glide licence and compute capacity are available.

9.6 Detach from and return to the run

Detach from **screen** without stopping the calculation:

```
Ctrl-A, then D
```

List and reattach to sessions:

```
screen -ls  
screen -r ahc_run
```

Do not use `screen -X -S <name> quit` unless the intention is to terminate the session and any foreground job running inside it.

9.7 Expected AHC outputs

The downstream pipeline expects the completed results at:

```
/home/andy/proj/work_results/AHC_Resultsbin/
├── SMILES-RNN_PRRun/
│   ├── scores.csv
│   ├── iterations/
│   │   ├── 000001_scores.csv
│   │   └── ...
│   └── PR_GlideDock/
│       └── <iteration>/<variant>_lib.sdfgz
└── SMILES-RNN_PPSrun/
    ├── scores.csv
    ├── iterations/
    │   ├── 000001_scores.csv
    │   └── ...
    └── PPS_GlideDock/
        └── <iteration>/<variant>_lib.sdfgz
```

The exact auxiliary files are controlled by the external SMILES-RNN/MolScore implementation. The minimum downstream requirements are the completed `scores.csv` files and, when cross-docking uses original AHC variants, the matching Glide pose hierarchy.

9.8 Monitoring an active AHC job

Once at least two iteration CSV files exist, the monitoring runners described in Section 11 can analyse the completed iterations while excluding the newest potentially active file. Monitoring reads the AHC outputs; it does not control, pause, or modify the AHC generation process.

10. Step 1: completed AHC analysis

```
conda activate analysisenv
cd /home/andy/proj/DProject_Workspace

python Modules/ACHresultspackage/CompletedAnalysis/run_analysis.py \
    --config Modules/ACHresultspackage/CompletedAnalysis/analysis.in
```

The current configuration enables:

```
physchem=on
last_window=on
scaffold=on
chemical_space=on
```

Processing follows a dependency-safe order:

```
PR/PPS score CSVs
  → physicochemical analysis
  → last-50-iteration comparison
  → joint chemical-space embeddings
  → scaffold and fingerprint diversity
```

The expected fingerprint UMAP cache is:

```
work_results/analysis/chemical_space/
└─ PR_PPS_completed_chemical_space/tables/joint_umap_fp.csv.gz
```

With `usecache=on`, a cache is reused only when the PR, PPS, and reference SHA-256 values and the calculation settings match. Use `usecache=off` when an input population, reference table, or embedding setting has changed.

The physicochemical figures use:

```
PPS: #D9534F (red)
PR:  #2878B5 (blue)
```

The reported descriptors are molecular weight, number of rotatable bonds, and calculated logP. The molecular-weight axis starts at zero and is shared between PR and PPS. Rotatable bonds use integer ticks from 0 to 17. OM is the default reference compound.

10.1 Fingerprint and scaffold-diversity calculations

The scaffold block uses canonical SMILES from the validated chemical-space membership table. RDKit is used to generate Morgan fingerprints and Bemis-Murcko scaffolds. The current fingerprint defaults are radius 2 and 16,384 bits.

For two binary fingerprints **A** and **B**, molecular similarity is evaluated with the Tanimoto coefficient:

$$T(A, B) = |A \cap B| / |A \cup B| = c / (a + b - c)$$

where **a** and **b** are the numbers of active bits in the two fingerprints and **c** is the number of active bits shared by both fingerprints.

The reported fingerprint-diversity value for a scope containing **M** unique molecular pairs is:

$$D_{FP} = 1 - \text{mean}(T) = 1 - (1/M) \sum_{i < j} T(A_i, A_j)$$

Larger values therefore indicate lower mean pairwise similarity and greater fingerprint diversity. The code also reports the median and 95th percentile of pairwise similarity and each molecule's nearest-neighbour similarity within the analysed scope.

For comparison with molecules generated in earlier iterations, the nearest prior-population similarity of molecule **i** is:

$$T_{i,prior}(max) = \max[j \text{ in prior}] T(A_i, A_j)$$

The analysis reports its mean and median and the fractions below 0.4 and 0.6. These thresholds describe fingerprint dissimilarity and should not be treated as universal definitions of chemical novelty.

If scaffold s occurs n_s times in a population of N molecules, its frequency is:

$$p_s = n_s / N$$

The scaffold Shannon entropy is:

$$H = -\sum[s] p_s \ln(p_s)$$

and the normalized entropy is:

$$H_{norm} = H / \ln(K)$$

where K is the number of unique scaffolds. The implementation reports zero normalized entropy when $K \leq 1$. A larger normalized entropy indicates a more even distribution across the observed scaffolds; it does not by itself imply greater biological relevance.

11. Optional: monitoring an active AHC run

```
python Modules/ACHresultspackage/Monitoring/run_monitor.py \
  --config Modules/ACHresultspackage/Monitoring/t_mp_pr.in

python Modules/ACHresultspackage/Monitoring/run_monitor.py \
  --config Modules/ACHresultspackage/Monitoring/t_mp_pps.in
```

Processing contract:

```
iterations/*_scores.csv
  → sort files
  → exclude newest file - to ensure removal of incomplete runs
  → merge remaining files - same compiling logic as original AHC logic
  → clean and canonicalise molecules - for downstream calculations
  → physicochemical figures - visual analysis
  → docking-score trajectory - visual analysis
```

Current thresholds are PR -6.0 and PPS -9.0 . If zero or one iteration CSV is found, the runner cannot create a safe merged monitoring input. For this study, at around 365 iterations, the model was stopped, but this may be different depending on the AHC restraint settings. so be sure to check the docking-score trajectory regularly to see how the model is doing.

12. Step 2: cross-docking calculation

```
conda activate analysisenv
cd /home/andy/proj/DProject_Workspace

python Modules/Crossdocking/run_crossdocking.py \
  --config Modules/Crossdocking/crossdocking.in
```

12.1 Population filtering and selection

Each AHC population is independently processed:

```
Raw AHC score table
  → retain valid and unique records
  → parse and canonicalise SMILES
  → require finite, nonzero own-state score
  → require a usable best-variant identifier
  → remove canonical duplicates
  → rank by own-state docking score
  → select the top percentage
  → annotate PAINS and BRENK alerts
```

Current production selection:

```
best_dscore_top_per=1
testmode_top_n=
```

The selected count is calculated independently for each population. If **N_usable** is the number of usable canonical molecules and **q** is the configured percentage, then:

```
N_selected = ceil(N_usable × q / 100)
```

The current data produce approximately 205 PPS and 197 PR molecules. **testmode_top_n** overrides percentage selection when it contains a value. PAINS and BRENK matches are annotations and are not automatically excluded.

12.2 Cross-docking design

```
L_PPS_to_R_PR: PPS-generated ligands → PR receptor grid
L_PR_to_R_PPS: PR-generated ligands → PPS receptor grid
```

The original AHC score supplies the own-state score. Cross-docking supplies the opposite-state score.

12.3 Ligand preparation

The current setting is:

```
ligand_prep_mode=on
```

LigPrep may generate multiple protonation states, tautomers, and stereoisomers. The workflow retains the most negative score over generated variants and poses. This may introduce unequal-enumeration bias because molecules with more variants receive more opportunities to obtain an extreme score. Interpretation must therefore consider `prepared_variant_count`.

With `ligand_prep_mode=off`, the original AHC-selected `*_lib.sdfgz` variant is reused. This provides a more direct same-variant receptor-state comparison.

12.4 Cross-docking outputs

```
work_results/crossdocking/
├── filtering/
│   ├── PPS_filtered.csv
│   ├── PPS_filtered_filterlog.txt
│   ├── PR_filtered.csv
│   └── PR_filtered_filterlog.txt
├── crossdocking/
│   ├── L_PPS_to_R_PR/
│   ├── L_PR_to_R_PPS/
│   ├── double_arm_manifest.csv
│   ├── double_arm_summary.csv
│   ├── double_arm_docking_scores.csv
│   └── double_arm_best_docking_scores.csv
├── pipeline_run_summary.json
└── crossdocking_calculation_manifest.json
```

Per-ligand job directories containing `input/`, `ligprep/`, docking CSVs, `docking_lib.sdfgz`, `docking.in`, and `glide_console.log` are expected.

13. Step 3: completed cross-docking analysis

When using an existing completed cross-docking calculation, run:

```
conda activate interanaly
cd /home/andy/proj/DProject_Workspace

python -m Modules.Crossdocking.CompletedAnalysis.crossdocking_analysis \
    --config Modules/Crossdocking/crossdocking.in
```

The module form is recommended because it reliably establishes the repository package root.

Current settings:

```
score_analysis=on
interaction_analysis=on
reference_interaction_analysis=on
interaction_mode=both
multiprocessing=on
posecheck_workers=30
posecheck_chunk_size=auto
prolif_workers=30
```

The tested multiprocessing settings have worked successfully, but may require communication with fellow students as this may reduce the server performance largely. It is recommended to use less than 30 workers.

13.1 Selectivity calculation

For a molecule generated for state **A**, let **S_{own}** be its original AHC docking score in the generation state and **S_{opp}** be its cross-docking score in the opposite receptor state. The selectivity margin is:

$$\Delta S_{sel} = S_{opp} - S_{own}$$

Because more negative Glide scores indicate more favourable predicted binding:

- a positive margin supports preference for the own state;
- a margin near zero indicates similar predicted docking;
- a negative margin indicates stronger predicted docking to the opposite state.

Current operational thresholds:

```
selectivity_threshold=2.0
strong_score_threshold=-8.0
```

These are prioritisation thresholds, not experimental evidence of biochemical selectivity.

With selectivity threshold $\tau = 2.0$ and strong-score threshold **S_{strong}** = -8.0, the principal classes are assigned as follows:

```
if  $\Delta S_{sel} \geq \tau$  and  $S_{own} \leq S_{strong}$ :
    class = selective own state
else if  $\Delta S_{sel} \geq \tau$  and  $S_{own} > S_{strong}$ :
    class = own-state preferred weak
else if  $\Delta S_{sel} \leq -\tau$  and  $S_{opp} \leq S_{strong}$ :
    class = reverse selective
else if  $\Delta S_{sel} \leq -\tau$  and  $S_{opp} > S_{strong}$ :
    class = reverse preferred weak
else if  $|\Delta S_{sel}| < \tau$  and both scores  $\leq S_{strong}$ :
    class = nonselective strong
else:
    class = inconclusive
```

Candidates are ranked first by this class priority, then by decreasing **ΔS_{sel}** , and finally by increasingly favourable (more negative) own-state score. If ligand-efficiency fields are available, the analogous margin is:

$$\Delta LE = LE_{opp} - LE_{own}$$

13.2 PoseCheck and ProLIF

With **posecheck_chunk_size=auto**, the chunk size for each population is:

```
ceil(valid_pose_count / posecheck_workers)
```

The minimum is one. If worker termination or memory pressure occurs, reduce `posecheck_workers` and `prolif_workers` before retrying.

Expected aggregate interaction results include:

```
work_results/analysis/crossdocking_interactions/CDres_LP_ON/  
├── populations/  
└── aggregate/  
    ├── interactions_long.csv  
    └── pose_metadata.csv
```

With `reference_interaction_analysis=on`, the pipeline also creates:

```
work_results/analysis/reference_interactions/CDres_LP_ON/  
└── reference_interactions.csv
```

The user does not prepare this CSV manually.

14. Step 4: structural, interaction-result, and figure pipeline

```
conda activate interanaly  
cd /home/andy/proj/DProject_Workspace  
  
python Modules/Analysis_Figen_block/run_analysis.py \  
    --config Modules/Analysis_Figen_block/analysis.in
```

Current top-level plan:

```
ahc_analysis=OFF  
structural_analysis=ON  
interaction_result_analysis=ON  
figures=ON
```

AHC analysis is disabled here because Step 1 runs it separately. Enabled modules execute in the fixed order AHC analysis, structural analysis, interaction-result analysis, and figures. A failure stops the sequence.

The parent `results_root` and `run_id` override the corresponding values in enabled child configurations.

Part IV. Computational module reference

15. Pocket-volume analysis

Purpose

The pocket-volume module recalculates the volume of the parts of selected Fpocket pockets associated with a reference ligand. It first selects alpha spheres located near ligand heavy atoms and then estimates the **union volume** occupied by the selected spheres using a bounding-box Monte Carlo method. Consequently, spatial overlap between spheres is counted only once.

Input and current settings

```
fpocket_dir=/home/andy/proj/work_results/fpocket_res/PPS_Protein_out
targets=2,38,51
ligand=/home/andy/proj/DProject_Workspace/AHC_Related/ref_structures/Structures/ligand_20W.pdb
distance_threshold=3.0
all_spheres=OFF
iterations=1000000
seed=42
chunk_size=100000
radius_offset=-1.6
```

The required inputs are:

- **fpocket_dir**: an Fpocket result directory containing a **pockets/** subdirectory and the requested **pocketN_vert.pqr** files;
- **targets**: comma-separated Fpocket pocket numbers;
- **ligand**: a PDB file containing the reference-ligand coordinates;
- **ligand_resname**: an optional residue-name filter when the ligand file is a complete protein-ligand complex rather than a ligand-only PDB;
- **all_spheres**: an option that bypasses the ligand-distance filter and uses every alpha sphere from the selected pockets.

Only ligand heavy atoms are used for proximity selection. If **ligand_resname** is empty, all heavy **ATOM** and **HETATM** coordinates in the supplied ligand PDB are treated as ligand coordinates. A residue-name filter should therefore be specified when a complete complex is used as the ligand input.

Calculation method

15.1 Alpha-sphere parsing

For each requested pocket, the module reads the alpha-sphere centre

```
c_i = (x_i, y_i, z_i)
```

and its raw radius **r_i** from **pocketN_vert.pqr**.

15.2 Ligand-proximity selection

Let **l_j** be the coordinate of ligand heavy atom **j**. Alpha sphere **i** is retained when its centre lies within the selected threshold **d** of at least one ligand heavy atom:

```
S_local = { i : min[j] ||c_i - l_j|| ≤ d }
```

The current default is $d = 3.0 \text{ \AA}$. This is a **centre-to-heavy-atom** distance criterion. It is not the distance between the ligand atom and the surface of the alpha sphere.

Sphere selection uses the original centre coordinates and does not depend on the radius correction applied during volume integration.

When `all_spheres=ON`, this proximity-selection step is bypassed and every alpha sphere from each selected pocket is included.

```
def select_ligand_local_spheres(spheres, ligand_coordinates, threshold):
    centres = np.asarray([s.centre for s in spheres], dtype=float)
    distance_squared = np.sum(
        (centres[:, None, :] - ligand_coordinates[None, :, :]) ** 2, axis=2
    )
    selected = np.any(distance_squared <= threshold**2, axis=1)
    return [s for s, keep in zip(spheres, selected) if keep]
```

15.3 Fpocket radius correction

The Fpocket volume implementation being reproduced calls `get_verts_volume_ptr(..., -1.6)`. To match this behaviour, the radius used for volume integration is:

$$r_i(\text{eff}) = r_i + \Delta r, \text{ where } \Delta r = -1.6 \text{ \AA}$$

Accordingly, the current configuration uses:

```
radius_offset=-1.6
```

Using the raw PQR radii without this correction can substantially overestimate the value corresponding to the current Fpocket calculation. A value of `radius_offset=0` should be used only when an uncorrected alpha-sphere union is intentionally required.

15.4 Bounding-box construction

The smallest axis-aligned box enclosing all selected, radius-corrected spheres is constructed as:

```
x_min = min[i](c_i,x - r_i(eff))
x_max = max[i](c_i,x + r_i(eff))
```

with equivalent expressions for `y` and `z`. The bounding-box volume is:

$$V_{\text{box}} = (x_{\text{max}} - x_{\text{min}})(y_{\text{max}} - y_{\text{min}})(z_{\text{max}} - z_{\text{min}})$$

15.5 Monte Carlo union-volume estimate

The module samples N uniformly distributed random points inside the bounding box. A sampled point p_k is classified as inside the pocket if it lies inside at least one selected sphere:

$$I_k = \begin{cases} 1 & \text{if at least one sphere } i \text{ satisfies} \\ & ||p_k - c_i||^2 \leq r_i(\text{eff})^2 \\ 0 & \text{otherwise} \end{cases}$$

The alpha-sphere union volume is estimated as:

$$\begin{aligned} N_{\text{inside}} &= \sum_{k=1}^N I_k \\ V_{\text{union(est.)}} &= V_{\text{box}} \times N_{\text{inside}} / N \end{aligned}$$

Because a sampled point is counted once even when it lies in several spheres, sphere overlap is not double-counted. This differs from the simple sum:

$$\sum [i] (4/3)\pi r_i(\text{eff})^3$$

which overestimates pocket volume whenever spheres overlap. The simple sum is reported as `naive_sphere_volume_sum_A3` for diagnostic purposes only; it is not the reported pocket-volume estimate.

15.6 Monte Carlo uncertainty

Writing

$$p_{\text{hat}} = N_{\text{inside}} / N$$

the reported sampling standard error is:

$$SE(V_{\text{est.}}) = V_{\text{box}} \times \sqrt{p_{\text{hat}}(1 - p_{\text{hat}}) / N}$$

Increasing `iterations` reduces the random sampling error. The current default uses 1,000,000 samples with seed 42. Per-pocket seeds are deterministically offset so different pockets do not reuse identical random streams.

`chunk_size` controls how many Monte Carlo points are processed at once. It affects memory usage and execution performance but does not change the intended estimator.

In Code these calculations were reproduced in

```
Modules/Analysis_Figen_block/structural_analysis/blocks/pocket_volume.py
```

```
def monte_carlo_union_volume(
    spheres, iterations, seed, chunk_size, radius_offset):
    centres = np.asarray([s.centre for s in spheres], dtype=float)
    raw_radii = np.asarray([s.radius for s in spheres], dtype=float)
```

```

radii = raw_radii + radius_offset          # Fpocket radius correction ( $\Delta r = -1.6 \text{ \AA}$ )

lower = np.min(centres - radii[:, None], axis=0)
upper = np.max(centres + radii[:, None], axis=0)
box_volume = float(np.prod(upper - lower))

rng = np.random.default_rng(seed)
points_inside, completed = 0, 0
radius_squared = radii**2
while completed < iterations:
    count = min(chunk_size, iterations - completed)
    points = rng.uniform(lower, upper, size=(count, 3))
    inside_any = np.zeros(count, dtype=bool)
    for centre, r2 in zip(centres, radius_squared):
        active = ~inside_any
        if not np.any(active):
            break
        delta = points[active] - centre
        inside_any[active] = np.einsum("ij,ij->i", delta, delta) <= r2
    points_inside += int(np.count_nonzero(inside_any))
    completed += count

fraction = points_inside / iterations
volume = box_volume * fraction
standard_error = box_volume * math.sqrt(fraction * (1 - fraction) / iterations)

return {"union_volume_A3": volume, "standard_error_A3": standard_error, ...}

```

15.7 Multiple target pockets

Each requested pocket is calculated independently. The module also generates a **combined** result by pooling ligand-local spheres from all requested pockets, removing exactly duplicated spheres, and calculating their union in a single bounding box.

Therefore:

$$V_{\text{combined}} \neq \sum [m] V_{\text{pocket},m}$$

when selected pockets overlap spatially. Overlap between different Fpocket pockets is counted only once in the combined result. If a target pocket has no sphere satisfying the ligand-distance threshold, its reported volume is zero.

Processing summary

```

Selected Fpocket spheres
→ identify spheres local to the ligand
→ apply the distance threshold
→ adjust sphere radii
→ sample the bounding region
→ count samples contained in pocket spheres
→ estimate occupied volume

```

Conceptually:

```
estimated volume = bounding-box volume × fraction of samples inside spheres
```

Outputs

The module writes matching CSV and JSON result tables under:

```
work_results/analysis/pocket_volume/<run_id>/
├ inputs/
├ intermediate/
├ tables/
│   ├── <output_prefix>.csv
│   └ <output_prefix>.json
├ logs/
└ run_manifest.json
```

The outputs report:

- individual requested-pocket and combined scopes;
- total and ligand-selected alpha-sphere counts;
- Monte Carlo union volume in Å³;
- sampling standard error in Å³;
- diagnostic naive effective-sphere volume sum;
- bounding-box limits and volume;
- numbers of sampled and inside points;
- distance threshold, radius offset, iteration count, and random seed.

User action required

`targets=2,38,51` is specific to the current `PPS_Protein_out` result. It must be reviewed whenever Fpocket is rerun, a receptor changes, or a different result directory is used. Pocket location and atoms must be inspected; the numeric ID alone does not guarantee pocket identity.

Interpretation and limitations

The reported quantity is a **ligand-associated alpha-sphere union volume**. It is not the physical volume occupied by the ligand, and it is not the strict geometric intersection between the pocket and a 3 Å shell around the ligand.

Once the centre of an alpha sphere passes the 3 Å proximity criterion, the complete radius-corrected sphere contributes to the union calculation.

The estimate inherits the geometric assumptions of Fpocket and should be used primarily for comparisons processed with identical settings. Differences in receptor preparation, ligand coordinates, Fpocket parameters, distance threshold, radius correction, target selection, or Monte Carlo settings can change the reported value.

16. Redocking validation

Input

```
pps_docking_csv=/home/andy/proj/work_results/Redocking/PPS_docking.csv
pps_rmsd_csv=/home/andy/proj/work_results/Redocking/PPS_RMSD.csv
pr_docking_csv=/home/andy/proj/work_results/Redocking/PR_docking.csv
```

```
pr_rmsd_csv=/home/andy/proj/work_results/Redocking/PR_RMSD.csv
score_ylim=-12,-1
rmsd_ylim=0,2.5
```

Processing and output

The module parses PPS and PR docking-score and RMSD results, constructs comparison tables, applies common plotting limits, and renders redocking validation figures. Plotting limits must be reviewed if new data fall outside the configured ranges.

For N matched redocked poses, the arithmetic means are calculated as:

$$\begin{aligned}\text{mean}(S) &= (1/N) \sum_{i=1 \text{ to } N} S_i \\ \text{mean}(R) &= (1/N) \sum_{i=1 \text{ to } N} R_i\end{aligned}$$

where S_i is the docking score and R_i is the RMSD of pose i . The module also reports the minimum, maximum, and median of both quantities.

A pose is counted as being within the redocking RMSD threshold when:

$$I_i(2 \text{ \AA}) = \begin{cases} 1 & \text{if } R_i < 2.0 \text{ \AA} \\ 0 & \text{if } R_i \geq 2.0 \text{ \AA} \end{cases}$$

The reported fraction is:

$$f(R < 2 \text{ \AA}) = (1/N) \sum_{i=1 \text{ to } N} I_i(2 \text{ \AA})$$

The docking-score and RMSD tables must have the same number of rows for each state, and the PPS and PR datasets must also contain equal pose counts. The implementation pairs their values by row order; the input CSVs must therefore use a consistent pose ordering.

17. Residue-hotspot analysis

Input

```
interactions_csv=/home/andy/proj/work_results/analysis/crossdocking_interactions/CDres_LP_0
N/aggregate/interactions_long.csv
denominators=PPS=205,PR=197
```

Processing

Before counting, repeated occurrences of the same population, molecule, and residue are deduplicated. Interaction-type prevalence additionally retains the interaction type in the deduplication key. Thus a molecule contributes no more than one count to a given residue or residue-interaction pair.

For population g and residue r , define:

```
I_m,r,g = 1  if molecule m interacts with residue r in population g
          0  otherwise
```

The residue prevalence is:

$$P_{r,g} = \sum[m] I_{m,r,g} / N_g$$

the execution of these calculations were conducted in
Modules/Analysis_Figen_block/interaction_result_analysis/blocks/residue_hotspot.py

```
residue = (residue_unique.groupby(["residue", "population_state"])["molecule_id"]
            .nunique().rename("poses").reset_index())
residue["prevalence"] = residue.apply(
    lambda row: row.poses / denominators[row.population_state], axis=1
)
```

where N_g is the configured denominator for population g .

For interaction type t , the type-specific prevalence is:

$$P_{r,t,g} = \sum[m] I_{m,r,t,g} / N_g$$

The dominant interaction type displayed for a residue and population is:

$$t^*_{r,g} = \arg \max[t] P_{r,t,g}$$

The denominators must be changed whenever selection percentage, filtering, pose availability, or the cross-docking result changes. They should represent the final analysed population, not an assumed count from a previous run.

Expected output tables:

```
work_results/analysis/residue_hotspot/
├── PR_PPS_analysis_residue_hotspot/tables/
│   ├── residue_prevalence.csv
│   └── interaction_type_prevalence.csv
```

The optional `full_last`, `top_candidates`, and `own_opposite` blocks require additional specialised inputs. Empty paths are acceptable only while the corresponding blocks remain disabled.

18. Figure generation

Current figure blocks:

```
ahc_physchem=ON
chemical_space=ON
physchem_last_window=ON
interaction_hotspot=ON
hotspot_full_last=OFF
hotspot_top_candidates=OFF
own_opposite=OFF
```

18.1 AHC physicochemical figures

```
ahc_input=/home/andy/proj/work_results/AHC_Resultsbin/SMILES-RNN_PPSrun/scores.csv
ahc_job_name=PPS
ahc_threshold=-9.0
descriptors=desc_MolWt,desc_NumRotatableBonds,desc_CLogP
```

The standalone configuration currently targets PPS. A PR-specific run requires changing `ahc_input`, `ahc_job_name`, and `ahc_threshold`.

18.2 Chemical-space figures

```
cache_path=/home/andy/proj/work_results/analysis/chemical_space/PR_PPS_completed_chemical_s
pace/tables/joint_umap_fp.csv.gz
```

The cache must first be produced by completed AHC analysis. A mismatched run ID causes `FileNotFoundError` even when another valid cache exists.

18.3 Last-window physicochemical comparison

```
last_n=50
pr_threshold=-6.0
pps_threshold=-9.0
```

This block compares the last 50 AHC iterations for PR and PPS.

18.4 Interaction-hotspot figure

Input contract:

```
residue_prevalence.csv
+ interaction_type_prevalence.csv
+ selected Fpocket pocket files
+ reference_interactions.csv
  → PPS/PR interaction-hotspot figure
```

Expected output:

```
work_results/Figs/interaction_hotspot/<run_id>/
├─ interaction_hotspot.png
├─ associated data table
└─ figure manifest
```

The required colour convention is PPS red (#D9534F) and PR blue (#2878B5).

Part V. Validation and review

19. Minimum validation checks

19.1 Completed AHC analysis

```
find /home/andy/proj/work_results/analysis/chemical_space \
  -type f -name 'joint_umap_fp*.csv.gz' | sort

find /home/andy/proj/work_results/Figs -type f | sort
```

Confirm that PR and PPS records are present, zero-score placeholders were removed, molecular properties are plausible, OM is present where expected, and the colour convention is correct.

19.2 Cross-docking

```
python -m json.tool \
  /home/andy/proj/work_results/crossdocking/pipeline_run_summary.json
```

Confirm that both arms are present, selected counts match the filtering logs, there are no unexplained unavailable or failed targets, and the final status is **completed**.

19.3 Interaction analysis

```
test -s
/home/andy/proj/work_results/analysis/crossdocking_interactions/CDres_LP_ON/aggregate/interactions_long.csv
test -s
/home/andy/proj/work_results/analysis/reference_interactions/CDres_LP_ON/reference_interactions.csv
test -s
/home/andy/proj/work_results/analysis/completed_crossdocking/CDres_LP_ON/crossdocking_analysis.json
```

19.4 Final visual audit

```
find /home/andy/proj/work_results/Figs \  
-type f \( -name '*.png' -o -name '*.pdf' \) | sort
```

Inspect every final figure for missing populations, incorrect colours, clipped labels, overlapping legends, empty axes, inconsistent ranges, incorrect pocket labels, and missing reference markers.

20. Reusing existing results

A reusable cross-docking directory should contain:

```
work_results/crossdocking/  
├─ filtering/  
├─ crossdocking/  
├─ pipeline_run_summary.json  
└─ crossdocking_calculation_manifest.json
```

Before reuse, verify input score identity, selected counts, grid identity, LigPrep mode, Glide precision, completion status, manifest, and exact run ID.

Use `resume=off` for a clean production run. Use `resume=on` only when intentionally reusing eligible existing results. Presence of a file alone does not establish that it matches the current configuration.

Where collision-safe figure naming is implemented, repeated outputs may be:

```
residue_hotspot_full_last50.png  
residue_hotspot_full_last50_1.png  
residue_hotspot_full_last50_2.png
```

Use the manifest and modification time to identify the output from the current run.

Part VI. Troubleshooting

21. Common problems

ModuleNotFoundError: No module named 'Modules'

Run from the repository root. For completed cross-docking analysis, use module execution:

```
cd /home/andy/proj/DProject_Workspace  
python -m Modules.Crossdocking.CompletedAnalysis.crossdocking_analysis \  
--config Modules/Crossdocking/crossdocking.in
```

BrokenProcessPool

Possible causes include an incorrect Conda environment, worker crash, memory exhaustion, too many workers, or a malformed pose triggering a native-library failure. Activate `interanaly`, confirm imports, and reduce worker counts before retrying.

Chemical-space cache not found

Locate the actual cache:

```
find /home/andy/proj/work_results/analysis/chemical_space \
  -type f -name 'joint_umap_fp*.csv.gz' | sort
```

Then update `cache_path` in `figures.in`. This commonly results from a mismatch between the completed-AHC and figure run IDs.

Monitoring merge failure

```
find <iterations-directory> \
  -maxdepth 1 -type f -name '*_scores.csv' | wc -l
```

At least two iteration files are required because the newest is excluded.

UMAP warnings

A warning that `n_jobs` is forced to one by a fixed random state is expected when reproducibility is enabled. A warning that inverse transformation is not available for Jaccard distance does not invalidate the forward embedding used for plotting.

MDSAnalysis bond warnings

MDSAnalysis may guess bonds from coordinates when explicit topology is absent. This does not necessarily mean failure, but unusual interaction assignments should be inspected carefully.

Empty interaction figures

Check, in order:

1. `interactions_long.csv` exists and is non-empty.
2. PPS and PR population labels are present.
3. Denominators match the analysed populations.
4. Residue-prevalence tables were generated.
5. `reference_interactions.csv` exists.
6. The figure block is enabled.
7. All configured pocket files exist.

Part VII. Reproducibility and reporting

22. Provenance record

For every final production run, record:

- repository commit ID;
- execution date and hostname;
- Python and Schrödinger versions;
- Conda environment export;

- all active `.in` files;
- input paths and SHA-256 checksums;
- run IDs and random seeds;
- population counts and worker settings;
- cache-use decision;
- calculation and analysis manifests.

```
cd /home/andy/proj/DProject_Workspace
git rev-parse HEAD
python --version
hostname
date --iso-8601=seconds

sha256sum \
  /home/andy/proj/work_results/AHC_Resultsbin/SMILES-RNN_PPSrun/scores.csv \
  /home/andy/proj/work_results/AHC_Resultsbin/SMILES-RNN_PRRun/scores.csv
```

Archive the configurations used for the final run:

```
mkdir -p /home/andy/proj/work_results/run_records/PR_PPS_final

cp Modules/ACHresultspackage/CompletedAnalysis/analysis.in \
  /home/andy/proj/work_results/run_records/PR_PPS_final/
cp Modules/Crossdocking/crossdocking.in \
  /home/andy/proj/work_results/run_records/PR_PPS_final/
cp Modules/Analysis_Figen_block/analysis.in \
  /home/andy/proj/work_results/run_records/PR_PPS_final/
```

23. Interpretation limitations

1. Glide scores are model-derived prioritisation measures, not experimental binding free energies.
2. A favourable selectivity margin does not prove biochemical state selectivity.
3. AHC populations depend on the upstream generation model and reward function.
4. Top-1% selection is conditioned on filtering, deduplication, and score availability.
5. PAINS and BRENK results are warnings, not automatic exclusions.
6. LigPrep enumeration may bias best-score comparisons when variant counts differ.
7. ProLIF results depend on pose geometry, structural preparation, and inferred bonding.
8. Pocket IDs are specific to an individual Fpocket run.
9. Population overlap and cross-docking selectivity answer different questions.
10. Final prioritisation should combine docking score, pose quality, interactions, chemical liabilities, and experimental evidence.

24. Items to finalise after the production server run

- Add exact package versions from the final Conda environments.
- Add explicit required-column tables based on the production `scores.csv`, redocking CSVs, and generated interaction tables.
- Record the final repository commit ID and configuration snapshots.
- Replace approximate expected molecule counts if production filtering changes.
- Record any intentional deviations from the settings documented above.

Closing note

I hope this document helps understand the code and file structures. In each of the code sub directories, individual readme files can be found which may also aid the understanding of the given section. Additional figures and interpretations are also stored in `/home/andy/proj/Presprep_addifigures` which were used for the presentation. which can also be found on the git repo.

I wish you the best.

Appendix. Standard execution sequence

```
cd /home/andy/proj/DProject_Workspace

# 1. Completed AHC analysis
conda activate analysisenv
python Modules/ACHresultspackage/CompletedAnalysis/run_analysis.py \
    --config Modules/ACHresultspackage/CompletedAnalysis/analysis.in

# 2. Cross-docking calculation
python Modules/Crossdocking/run_crossdocking.py \
    --config Modules/Crossdocking/crossdocking.in

# 3. Completed cross-docking analysis
conda activate interanaly
python -m Modules.Crossdocking.CompletedAnalysis.crossdocking_analysis \
    --config Modules/Crossdocking/crossdocking.in

# 4. Structural, interaction-result, and figure analysis
python Modules/Analysis_Figen_block/run_analysis.py \
    --config Modules/Analysis_Figen_block/analysis.in
```

Optional monitoring:

```
conda activate analysisenv
python Modules/ACHresultspackage/Monitoring/run_monitor.py \
    --config Modules/ACHresultspackage/Monitoring/t_mp_pr.in
python Modules/ACHresultspackage/Monitoring/run_monitor.py \
    --config Modules/ACHresultspackage/Monitoring/t_mp_pps.in
```